Vitis High-Level Synthesis User Guide (UG1399)

Confidential - Copyright © Fluid Topics

Top-Level Performance Pragma

Top-Level Performance Pragma

Top-Level Performance Pragma

The `top-level performance pragma` provides a mechanism to define a performance goal assigned to the top of the design. It constitutes a throughput constraint that acts as a high-level directive to guide the synthesis tool in optimizing the design relative to the goal.
With the top-level performance pragma, the compiler conducts a design-wide throughput analysis. It evaluates several pragmas for inference to determine the feasibility of meeting the throughput goal. Based on its analysis, the tool automatically infers appropriate loop-level performance pragma for individual loops and loop nests throughout the design hierarchy.

Loop-Level Performance Pragma

Loop-level performance pragmas apply to individual loops to explicitly guide the tool. These pragmas can be automatically inferred and calibrated by the top-level performance pragma algorithm, or they can be manually applied by the user. They enable fine-grained performance transformation by automatically inferring one or more classic low-level pragmas such as pipeline, unroll, flatten, reshape, and partition.
Benefits

- Precise Control of Loop Behavior: Loop-level pragmas provide designers with the ability to optimize specific loops for throughput, enabling greater control over the performance of critical paths.
- Support for Top-Down Goals: When used in conjunction with the top-level performance pragma, loop-level pragmas are the concrete knobs that help close the gap between current performance and the specified system-level targets.

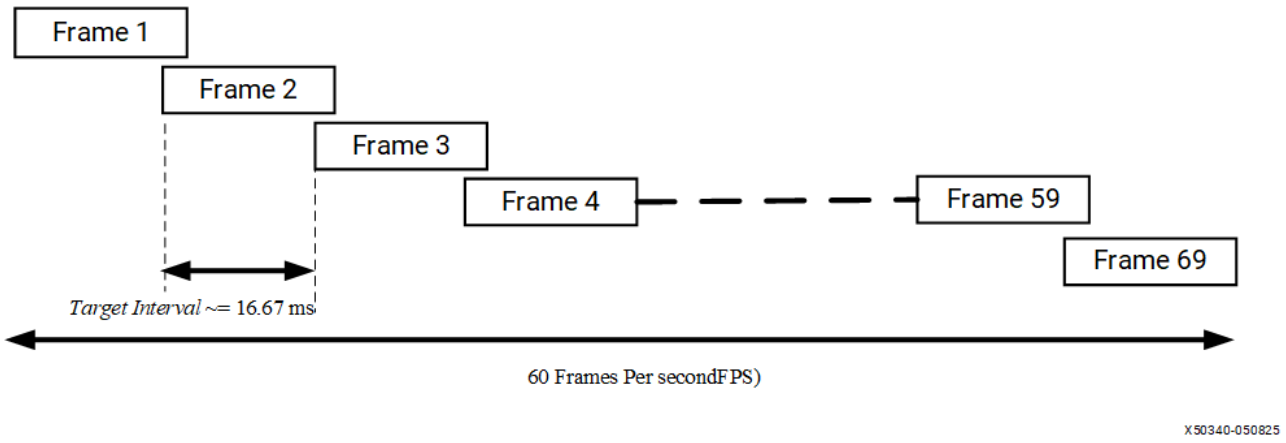
Methodology of Performance Pragma

Top-level performance pragma methodology is a streamlined approach to guide the optimization process and achieve demanding bandwidth goals.

Step 1: Calculate the performance target base on the throughput goal

Calculating the performance target for a video application

- Let's assume an image processing design targeting a frame rate of 60 frames per second (60 fps).
- **Step 1: Determine the total number of frames**
  - Total number of frames per sec = 60.
- **Step 2: Determine the top-level performance target (target_ti) :**
  - To achieve a 60 FPS frame rate, the function top must be ready to restart and read a new frame within 1/60th of a second.
  - target_ti = 1/FPS = 1/60 ~= 16.67 milliseconds
  - This means that the function top must be ready to restart and read a new frame within 16.7 milliseconds to maintain the 60 FPS target.



Step 3: Re-architect the code for dataflow

Before applying performance pragmas and diving into any Vitis HLS optimizations, it's important to ensure the code is structured for efficient hardware implementation. This involves creating tasks (loop and function) in accordance with a canonical dataflow form. This will create a solid foundation for Vitis HLS to effectively optimize the code and implement parallelism.

Step 4: Run CSIM and determine loop trip counts

- The next step in the methodology is to run C-simulation. During C-simulation, enable the loop tripcount profiling option to automatically capture actual loop iteration counts. This data is essential because the performance pragma algorithm must accurately budget all loops in the design; precise tripcounts prevent reliance on default assumptions and improve the fidelity of performance estimates. During C-simulation, enable the loop tripcount profiling option to automatically capture actual loop iteration counts. That metric is important since the performance pragma algorithm needs to precisely budget all loops in the design. By default, performance pragma estimates the loop bound for variable loops to be “1024”, which could cause an inaccurate estimation of the performance for loops whose bound are greater than 1024.
- Refer to the CSIM Loop Tripcount Option.

Step 5: Add the top-Level performance target

- The next step in the methodology is to apply the performance target. Using the `target_ti` parameter (target interval), specify your desired performance goal directly within your code, as shown below. To meet a frame rate of 60 fps, your `target_ti` would be 16.7 milliseconds.

```
#pragma HLS performance target_ti = <> ms/cycles
```

Step 6: Identify bottleneck loops (if any)

- **Run C Synthesis:** After applying the top-level performance pragma, run C synthesis in Vitis HLS.
- **Analyze the Report:** Carefully examine the C synthesis report. Identify loops/functions that don't meet the target_ti requirement

Step 7: Add/Update local performance targets

- For critical loops identified as bottlenecks, specify loop-level performance targets using the same `target_ti` parameter. This directs Vitis HLS in focusing optimization efforts on these loops.
- Rerun C synthesis and analysis of the updated reports. Continue refining loop-level targets until your design meets the overall performance goal.
- By following this iterative process of analysis and refinement using performance pragmas, you can guide Vitis HLS to achieve optimal performance in your hardware designs.

Understanding Top-Level Performance Pragma's Optimization Strategy

Performance Pragma: Prioritization of Timing Constraints
The performance pragma's primary optimization goal is to meet the specified timing requirements. It prioritizes achieving timing constraints and will not sacrifice timing constraints in an attempt to meet performance targets. Therefore, even if performance targets are not fully achieved, the pragma will ensure that the design meets its timing requirements. If desired performance targets are not met, it is recommended to use more granular, classic pragmas such as `unroll` and `pipeline` to further enhance performance without violating the established timing constraints.
Re-architecting Code for Top-Level Performance Pragma optimization

- To effectively utilize the top-level performance pragma, the design necessitates a re-architecture into the Load-compute-store (LCS) paradigm and employ dataflow pragma.

Dynamic trip count information

- Enable the loop tripcount profiling option during C-simulation to automatically capture actual loop iteration counts. Without the loop tripcount, performance pragma estimates the loop bound for dynamic loops to be “1024”, which could cause an inaccurate estimation of the performance for loops whose bound are greater than 1024.
- For variable loops, users should provide dynamic trip count information by using HLS trip count pragma (pragma HLS loop_tripcount max=N)

Locking in Inferred Pragmas

Export the low-level pragmas inferred by Performance Pragmas, along with any manually specified pragmas, to a user configuration file. Re-inject these pragmas into your source or directives file to lock in the selected optimizations. This prevents the performance budgeter from re-exploring pragma combinations during subsequent runs, reducing extra cycles and shortening iteration time.

Limitations

Pragma precedence
The following guidelines outline the precedence of various pragmas when applied to loops and arrays:

- **PIPIPELINE OFF Pragma**:
  - When the `PIPIPELINE OFF` pragma is applied to a loop, the `PIPELINE` pragma will not be automatically inferred for that loop by the 'Loop Level Performance pragma'
- **UNROLL OFF Pragma**:
  - Applying the `UNROLL OFF` pragma to a loop prevents any `UNROLL` pragma from being inferred for that loop by the 'Loop Level Performance pragma'
- **FLATTEN Pragma**:
  - If the `FLATTEN` pragma is used on a loop, it will not allow any other `FLATTEN` pragma to be inferred by the 'Loop Level Performance pragma'
- **ARRAY_PARTITION OFF Pragma**:
  - When `ARRAY_PARTITION OFF` is applied to a local array, this will prevent any `ARRAY_PARTITION` pragma from being inferred on that array by the 'Loop Level Performance pragma'

Interface Port Limitation

- By default, arrays at the interface of the top function are not automatically inferred with the `ARRAY_PARTITION` pragma. This could lead to performance bottlenecks. Users can address this issue by enabling the feature with the following command:

```
config_array_partition -throughput_driven=aggressive
```

Unsupported Libraries
The top-level performance pragma utilizes code analyzer technology to conduct a design-wide performance analysis, enabling the achievement of this goal. The limitations associated with code analyzers also apply to the top-level performance pragma. The following lists the Behaviour/limitations

Features	Behavior/Limitation
ap_cint	The tool exits with an explicit warning message
ap_(u)int / ap_(u)fixed	Performance models are inaccurate
Big constant arrays of ap_int / ap_fixed	Using them with macro NON_C99STRING will result in a compilation error
std::complex<ap_fixed>	Lead to a compiler error on Windows
HLS IP blocks (FFT, FIR, …), hls_math.h, ap_wait, hls::vector, ap_axis/ap_axiu	Performance models are inaccurate
hls::stream_of_blocks, hls::task, hls::split / hls::merge, hls::print, hls::half, ap_utils.h, hls_fpo.h, ap_float, RTL Blackboxes, hls::burst_maxi, hls::fence, hls::directio	The tool exits with an explicit warning message
Deprecated HLS pragmas	Generates a warning (the deprecated pragma is ignored)
Function pipeline with sub-loop	Assertion failure

Known Issues

Long Compile time

- The top-level performance pragma performs a comprehensive, design-wide performance analysis. This process can be compared to finding the best route from point A to point B among numerous possibilities. By exploring various optimization strategies across the entire design, the pragma aims to identify the most efficient optimizations to meet performance targets. However, this extensive exploration phase, while crucial for achieving optimal results, can sometimes, lead to longer compilation times. If you encounter an issue, please contact AMD support.

Aggressive infered performance targets

- The top-level performance pragma aims for optimal performance with minimal resource usage through design-wide analysis. Be aware that for specific loops, the tool's automated optimization might suggest aggressive targets, potentially leading to excessive resource consumption. If particular loops exhibit high resource utilization, using classic pragmas offers finer-grained control over resources.